

LABORATORY OBSERVATION / RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 1

Student Name: K. Sirisha

Roll No.: A24126552260

Section: B

EXPERIMENT 1 - INTERACTIVE PRODUCT CALCULATOR (CO-1)

1. TITLE

Develop an Interactive Product Price Calculator Webpage Using HTML, CSS, JavaScript, and DOM Event Listeners.

2. AIM

To design and implement an interactive webpage containing product name, unit price, and quantity inputs that dynamically calculates and renders the subtotal, tax (18% GST), and total order price in real time upon any input modification, without refreshing the webpage.

3. OBJECTIVE

- To create a structured, accessible HTML5 user interface using semantic form controls.
- To capture real-time user input events using JavaScript `addEventListener` (input and change events).
- To perform arithmetic operations including subtotal calculation, percentage tax computation, and grand total evaluation.
- To dynamically manipulate DOM tree elements using `textContent` so that calculated financial values are reflected instantaneously without a page reload.
- To implement input validation ensuring only positive numerical values are processed, and to display user-friendly error messages for invalid states.
- To provide quick preset options and a form reset mechanism.

4. THEORY

Dynamic web applications rely on client-side scripting to manipulate the Document Object Model (DOM) without triggering a traditional full-page HTTP reload.

1. The Document Object Model (DOM)

The DOM is an object-oriented representation of the webpage in which HTML elements are modelled as nodes with configurable properties, methods, and event handlers.

2. DOM Selection Methods

- `document.getElementById(id)` - returns the Element object matching the specified unique ID.
- `document.querySelectorAll(selector)` - returns a static NodeList of all matching elements.

3. Event-Driven Architecture and Event Listeners

- The modern W3C standard binds events using `element.addEventListener('eventType', handler)`.
- **input event:** fires whenever the value of an `<input>`, `<select>` or `<textarea>` element is altered. It gives instantaneous responsiveness on keystrokes, spinner dragging, or pasting text.

- **change event:** fires when an alteration is committed by the user, typically on losing focus or on selecting a value from a dropdown.

4. Data Type Conversion in JavaScript

- Values retrieved from HTML form inputs are always returned as strings, even when `<input type="number">` is used.
- `parseFloat(string)` parses the textual argument and returns a floating-point number.
- `parseInt(string, radix)` parses a string and returns an integer in the specified radix (base 10).
- `isNaN(value)` determines whether a value is Not-a-Number.

5. DOM Updating Without Refreshing

Updating `element.textContent` alters the text node of an element immediately inside the browser layout engine. This does not trigger a page refresh, so the application state is preserved and the interface remains seamless.

6. Currency Localization

`Number.prototype.toLocaleString('en-IN', options)` formats numbers with currency symbols, decimal places, and the regional numbering system (Indian lakh/crore comma grouping).

5. ALGORITHM / PROCEDURE

1. Create a root HTML5 document (`index.html`) declaring the standard meta tags and stylesheet link.
2. Structure a main container card with an input form containing Product Name, Unit Price (₹), and Quantity fields.
3. Add placeholder `<span>` tags below each input to render validation error notices.
4. Construct an Invoice Summary card with dedicated element IDs for `summaryName`, `summaryPrice`, `summaryQuantity`, `summarySubtotal`, `summaryTax`, and `summaryTotal`.
5. Add action buttons for Reset Form and the quick-preset load buttons.
6. Style the layout using CSS (`style.css`) with CSS Grid/Flexbox, distinct card boundaries, clean typography, responsive breakpoints, and an animated status pulse.
7. Create the JavaScript file (`script.js`) and bind a `DOMContentLoaded` listener to ensure safe DOM element querying.
8. Implement the `calculateTotal()` function:
 - Extract the string values from the input fields.
 - Validate that the product name is non-empty.
 - Validate that price is numeric and greater than or equal to zero.
 - Validate that quantity is an integer greater than or equal to one.
 - If invalid, apply the `.invalid` CSS class, render an error message, and zero the total.
 - If valid, compute `subtotal = unitPrice x quantity`, `tax = subtotal x 0.18`, and `grandTotal = subtotal + tax`.
 - Format the computed values into Indian Rupee notation and update the summary text nodes.
9. Attach input and change event listeners to the price and quantity fields.
0. Attach click handlers to the preset buttons and the reset button.
1. Load the page in a web browser and verify dynamic recalculation without flickering or page reload.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Interactive Product Calculator</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <header class="header">
      <h1>Interactive Product Calculator</h1>
      <p class="subtitle">Full Stack Web Development (23CM4121) - CO-1 Task Demonstration</p>
    </header>

    <main class="calculator-layout">
      <!-- Product Input Form -->
      <section class="card form-card">
        <h2>Product Details</h2>
        <form id="productForm" onsubmit="return false;">
          <div class="form-group">
            <label for="productName">Product Name</label>
            <input type="text" id="productName" placeholder="e.g. Ergonomic Keyboard"
              value="Wireless Gaming Mouse">
            <span class="error-msg" id="nameError"></span>
          </div>

          <div class="form-group">
            <label for="productPrice">Unit Price (Rs.)</label>
            <input type="number" id="productPrice" placeholder="0.00" min="0" step="0.01"
              value="1499.00">
            <span class="error-msg" id="priceError"></span>
          </div>

          <div class="form-group">
            <label for="productQuantity">Quantity</label>
            <input type="number" id="productQuantity" placeholder="1" min="1" step="1" value="2">
            <span class="error-msg" id="qtyError"></span>
          </div>

          <div class="preset-section">
            <label class="preset-label">Quick Presets:</label>
            <div class="preset-buttons">
              <button type="button" class="preset-btn" data-name="Mechanical Keyboard"
                data-price="3499.00" data-qty="1">Keyboard</button>
              <button type="button" class="preset-btn" data-name="Wireless Mouse"
                data-price="1499.00" data-qty="2">Mouse</button>
              <button type="button" class="preset-btn" data-name="USB-C Hub"
                data-price="2750.00" data-qty="3">USB Hub</button>
            </div>
          </div>

          <div class="form-actions">
            <button type="button" id="resetBtn" class="btn btn-secondary">Reset Form</button>
          </div>
        </form>
      </section>

      <!-- Dynamic Invoice Summary Card -->
      <section class="card summary-card">
        <h2>Live Invoice Summary</h2>
        <div class="invoice-details">
          <div class="summary-row">
            <span class="label">Product Name:</span>
            <span class="value" id="summaryName">Wireless Gaming Mouse</span>
          </div>
          <div class="summary-row">
            <span class="label">Unit Price:</span>
            <span class="value" id="summaryPrice">&#8377;1,499.00</span>
          </div>
          <div class="summary-row">
            <span class="label">Quantity:</span>
            <span class="value" id="summaryQuantity">2</span>
          </div>
          <div class="summary-row">
            <span class="label">Subtotal:</span>
            <span class="value" id="summarySubtotal">&#8377;2,998.00</span>
          </div>
          <div class="summary-row">
            <span class="label">GST (18%):</span>
            <span class="value" id="summaryTax">&#8377;539.64</span>
          </div>
          <hr class="divider">
          <div class="summary-row total-row">
            <span class="label">Grand Total:</span>
            <span class="value total-amount" id="summaryTotal">&#8377;3,537.64</span>
          </div>
        </div>
        <div class="live-indicator">
          <span class="pulse-dot"></span>
          <span id="statusText">Dynamically Calculated (No Page Reload)</span>
        </div>
      </section>
    </main>
  </div>
```

```
<script src="script.js"></script>
</body>
</html>
```

style.css

```
* { margin: 0; padding: 0; box-sizing: border-box;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; }

body { background: #f0f4f8; color: #2d3748; padding: 30px 20px;
  display: flex; justify-content: center; }

.container { max-width: 920px; width: 100%; }

.header { text-align: center; margin-bottom: 28px; }
.header h1 { color: #1a365d; font-size: 28px; margin-bottom: 6px; }
.subtitle { color: #718096; font-size: 14px; }

.calculator-layout { display: grid; grid-template-columns: 1fr 1fr; gap: 24px; }
@media (max-width: 768px) { .calculator-layout { grid-template-columns: 1fr; } }

.card { background: #ffffff; border-radius: 12px; padding: 24px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08); border: 1px solid #e2e8f0; }
.card h2 { font-size: 20px; color: #2b6cb0; margin-bottom: 20px;
  border-bottom: 2px solid #ebf8ff; padding-bottom: 8px; }

.form-group { margin-bottom: 16px; }
.form-group label { display: block; font-size: 14px; font-weight: 600;
  margin-bottom: 6px; color: #4a5568; }
.form-group input { width: 100%; padding: 10px 14px; border: 1.5px solid #cbd5e0;
  border-radius: 6px; font-size: 15px; outline: none;
  transition: border-color 0.2s, box-shadow 0.2s; }
.form-group input:focus { border-color: #3182ce; box-shadow: 0 0 3px rgba(49,130,206,0.15); }
.form-group input.invalid { border-color: #e53e3e; background-color: #fff5f5; }

.error-msg { display: block; color: #e53e3e; font-size: 12px;
  margin-top: 4px; min-height: 16px; }

.preset-section { margin: 16px 0; }
.preset-label { font-size: 13px; font-weight: 600; color: #718096;
  display: block; margin-bottom: 6px; }
.preset-buttons { display: flex; gap: 8px; flex-wrap: wrap; }
.preset-btn { background: #edf2f7; border: 1px solid #cbd5e0; color: #4a5568;
  padding: 6px 12px; border-radius: 6px; font-size: 13px;
  cursor: pointer; transition: all 0.2s; }
.preset-btn:hover { background: #e2e8f0; color: #2d3748; }

.btn { padding: 10px 18px; border-radius: 6px; font-size: 14px; font-weight: 600;
  cursor: pointer; border: none; transition: all 0.2s; }
.btn-secondary { background: #edf2f7; color: #4a5568; border: 1px solid #cbd5e0; width: 100%; }
.btn-secondary:hover { background: #e2e8f0; }

.invoice-details { margin-bottom: 20px; }
.summary-row { display: flex; justify-content: space-between; align-items: center;
  padding: 10px 0; font-size: 15px; }
.summary-row .label { color: #718096; }
.summary-row .value { font-weight: 600; color: #2d3748; }
.divider { border: 0; border-top: 1px solid #e2e8f0; margin: 10px 0; }
.total-row { font-size: 18px; padding-top: 12px; }
.total-amount { color: #2b6cb0; font-size: 22px; }

.live-indicator { display: flex; align-items: center; background: #ebf8ff;
  padding: 10px 14px; border-radius: 8px; color: #2b6cb0;
  font-size: 13px; font-weight: 500; }
.pulse-dot { width: 10px; height: 10px; background: #38a169; border-radius: 50%;
  margin-right: 10px; animation: pulse 1.8s infinite; }

@keyframes pulse {
  0% { transform: scale(0.95); box-shadow: 0 0 0 0 rgba(56,161,105,0.7); }
  70% { transform: scale(1); box-shadow: 0 0 0 8px rgba(56,161,105,0); }
  100% { transform: scale(0.95); box-shadow: 0 0 0 0 rgba(56,161,105,0); }
}
```

script.js

```
/** Interactive Product Calculator
 * Demonstrates DOM Manipulation, Event Listeners, and Dynamic Price Calculation */

document.addEventListener('DOMContentLoaded', () => {

  // 1. DOM Element References
  const productNameInput = document.getElementById('productName');
  const unitPriceInput   = document.getElementById('productPrice');
  const quantityInput    = document.getElementById('productQuantity');
  const resetBtn         = document.getElementById('resetBtn');
  const presetBtns       = document.querySelectorAll('.preset-btn');

  // 2. Summary Card DOM References
  const summaryName      = document.getElementById('summaryName');
  const summaryPrice     = document.getElementById('summaryPrice');
  const summaryQuantity  = document.getElementById('summaryQuantity');
  const summarySubtotal  = document.getElementById('summarySubtotal');
  const summaryTax       = document.getElementById('summaryTax');
  const summaryByTotal   = document.getElementById('summaryTotal');
  const statusText       = document.getElementById('statusText');

  // 3. Error Feedback DOM References
```

```

const nameError = document.getElementById('nameError');
const priceError = document.getElementById('priceError');
const qtyError = document.getElementById('qtyError');

// 4. Helper Function: Currency Formatter
const formatCurrency = (amount) => {
  return '\u20B9' + amount.toLocaleString('en-IN', {
    minimumFractionDigits: 2,
    maximumFractionDigits: 2
  });
};

// 5. Dynamic Calculation and DOM Update Function
function calculateTotal() {
  const name = productNameInput.value.trim();
  const price = parseFloat(unitPriceInput.value);
  const qty = parseInt(quantityInput.value, 10);
  let isValid = true;

  // Validation for Product Name
  if (!name) {
    nameError.textContent = 'Please enter a product name.';
    productNameInput.classList.add('invalid');
    isValid = false;
  } else {
    nameError.textContent = '';
    productNameInput.classList.remove('invalid');
  }

  // Validation for Unit Price
  if (isNaN(price) || price < 0) {
    priceError.textContent = 'Price must be a valid positive number.';
    unitPriceInput.classList.add('invalid');
    isValid = false;
  } else {
    priceError.textContent = '';
    unitPriceInput.classList.remove('invalid');
  }

  // Validation for Quantity
  if (isNaN(qty) || qty < 1) {
    qtyError.textContent = 'Quantity must be at least 1.';
    quantityInput.classList.add('invalid');
    isValid = false;
  } else {
    qtyError.textContent = '';
    quantityInput.classList.remove('invalid');
  }

  // DOM Update Branch
  if (isValid) {
    const subtotal = price * qty;
    const tax = subtotal * 0.18; // 18% GST calculation
    const grandTotal = subtotal + tax;

    // Dynamically update text nodes without refreshing the webpage
    summaryName.textContent = name;
    summaryPrice.textContent = formatCurrency(price);
    summaryQuantity.textContent = qty;
    summarySubtotal.textContent = formatCurrency(subtotal);
    summaryTax.textContent = formatCurrency(tax);
    summaryTotal.textContent = formatCurrency(grandTotal);

    statusText.textContent = 'Dynamically Calculated (No Page Reload)';
    statusText.style.color = '#2b6cb0';
  } else {
    // Guard state on invalid input
    summaryTotal.textContent = '\u20B90.00';
    statusText.textContent = 'Invalid inputs detected. Please correct errors.';
    statusText.style.color = '#e53e3e';
  }
}

// 6. Attach Event Listeners for Dynamic Real-Time Updates
productNameInput.addEventListener('input', calculateTotal);
unitPriceInput.addEventListener('input', calculateTotal);
quantityInput.addEventListener('input', calculateTotal);

// Also attach 'change' event to handle number spinner button interactions
unitPriceInput.addEventListener('change', calculateTotal);
quantityInput.addEventListener('change', calculateTotal);

// 7. Preset Selection Handler
presetBtns.forEach(btn => {
  btn.addEventListener('click', () => {
    productNameInput.value = btn.dataset.name;
    unitPriceInput.value = btn.dataset.price;
    quantityInput.value = btn.dataset.qty;
    calculateTotal();
  });
});

// 8. Form Reset Handler
resetBtn.addEventListener('click', () => {
  productNameInput.value = 'Wireless Gaming Mouse';
  unitPriceInput.value = '1499.00';
  quantityInput.value = '2';
  nameError.textContent = '';
  priceError.textContent = '';
  qtyError.textContent = '';
  productNameInput.classList.remove('invalid');

```

```
        unitPriceInput.classList.remove('invalid');
        quantityInput.classList.remove('invalid');
        calculateTotal();
    });

    // Run initial calculation to populate default summary
    calculateTotal();
});
```

7. CODE EXPLANATION

Code	Explanation
<input type="number">	HTML5 numeric input field configured with min and step attributes for numeric entry.
document.addEventListener('DOMContentLoaded', ...)	Ensures the JavaScript executes only after the entire HTML document object model has been parsed into memory.
addEventListener('input', ...)	Event listener that triggers instantly on any user keystroke, character change, or deletion.
parseFloat(value)	Converts a string representation of a number into a 64-bit floating-point number.
parseInt(value, 10)	Parses a string value into a base-10 integer, used for discrete quantities.
isNaN(value)	Built-in function that checks whether a parsed value is mathematically invalid (Not-a-Number).
element.textContent	Safely gets or sets the raw text content of an element without HTML parsing overhead or XSS risk.
classList.add('invalid')	Programmatically appends the CSS error class to highlight offending input fields with red borders.
classList.remove('invalid')	Removes the CSS error class once valid data has been entered.
toLocaleString('en-IN')	Converts raw numeric values into standard Indian Rupee format with two decimal places.
dataset.name, dataset.price	Reads custom data-* attributes from the preset buttons to populate form fields dynamically.
onsubmit="return false;"	Prevents the default browser form submission that would cause an unwanted page reload.

8. TEST CASES AND EXECUTION DATA

The following test cases were recorded after executing the webpage in a browser.

Test Case	TC-01
Test / Input	Initial page load with default form values (Wireless Gaming Mouse, Price: 1499.00, Qty: 2).
Expected Result	The invoice summary displays Subtotal: ₹2,998.00, GST (18%): ₹539.64, and Grand Total: ₹3,537.64 dynamically.
Actual Result	Page loaded and automatically rendered all summary fields and the active pulse indicator without errors.
Status	Pass

TC-01 Output:

Interactive Product Calculator

Full Stack Web Development (23CM4121) - CO-1 Task Demonstration

Product Details

Product Name

Wireless Gaming Mouse

Unit Price (Rs.)

1499.00

Quantity

2

Quick Presets:

Keyboard

Mouse

USB Hub

Reset Form

Live Invoice Summary

Product Name:

Wireless Gaming Mouse

Unit Price:

₹1,499.00

Quantity:

2

Subtotal:

₹2,998.00

GST (18%):

₹539.64

Grand Total:

₹3,537.64

Dynamically Calculated (No Page Reload)

Test Case	TC-02
Test / Input	Dynamic quantity change: increase quantity from 2 to 5 using keyboard input.
Expected Result	Subtotal recalculates immediately to ₹7,495.00, GST to ₹1,349.10, and Grand Total to ₹8,844.10 without page reload.
Actual Result	The total amount updated instantly on keystroke with zero screen refresh or latency.
Status	Pass

TC-02 Output:

Interactive Product Calculator

Full Stack Web Development (23CM4121) - CO-1 Task Demonstration

Product Details

Product Name

Wireless Gaming Mouse

Unit Price (Rs.)

1499.00

Quantity

5

Quick Presets:

Keyboard

Mouse

USB Hub

Reset Form

Live Invoice Summary

Product Name:

Wireless Gaming Mouse

Unit Price:

₹1,499.00

Quantity:

5

Subtotal:

₹7,495.00

GST (18%):

₹1,349.10

Grand Total:

₹8,844.10

Dynamically Calculated (No Page Reload)

Test Case	TC-03
Test / Input	Negative / zero input validation: enter Quantity = 0 or -2.
Expected Result	A red border appears around the quantity input, the error message “Quantity must be at least 1.” is shown, and the total resets to ₹0.00.
Actual Result	Input highlighted in red, descriptive error string rendered, and calculations safely guarded.
Status	Pass

TC-03 Output:

Interactive Product Calculator

Full Stack Web Development (23CM4121) - CO-1 Task Demonstration

Product Details

Product Name

Wireless Gaming Mouse

Unit Price (Rs.)

1499.00

Quantity

0

Quantity must be at least 1.

Quick Presets:

Keyboard

Mouse

USB Hub

Reset Form

Live Invoice Summary

Product Name:

Wireless Gaming Mouse

Unit Price:

₹1,499.00

Quantity:

2

Subtotal:

₹2,998.00

GST (18%):

₹539.64

Grand Total:

₹0.00

Invalid inputs detected. Please correct errors.

9. OUTPUT

Output 1: Interactive Product Calculator Webpage Output

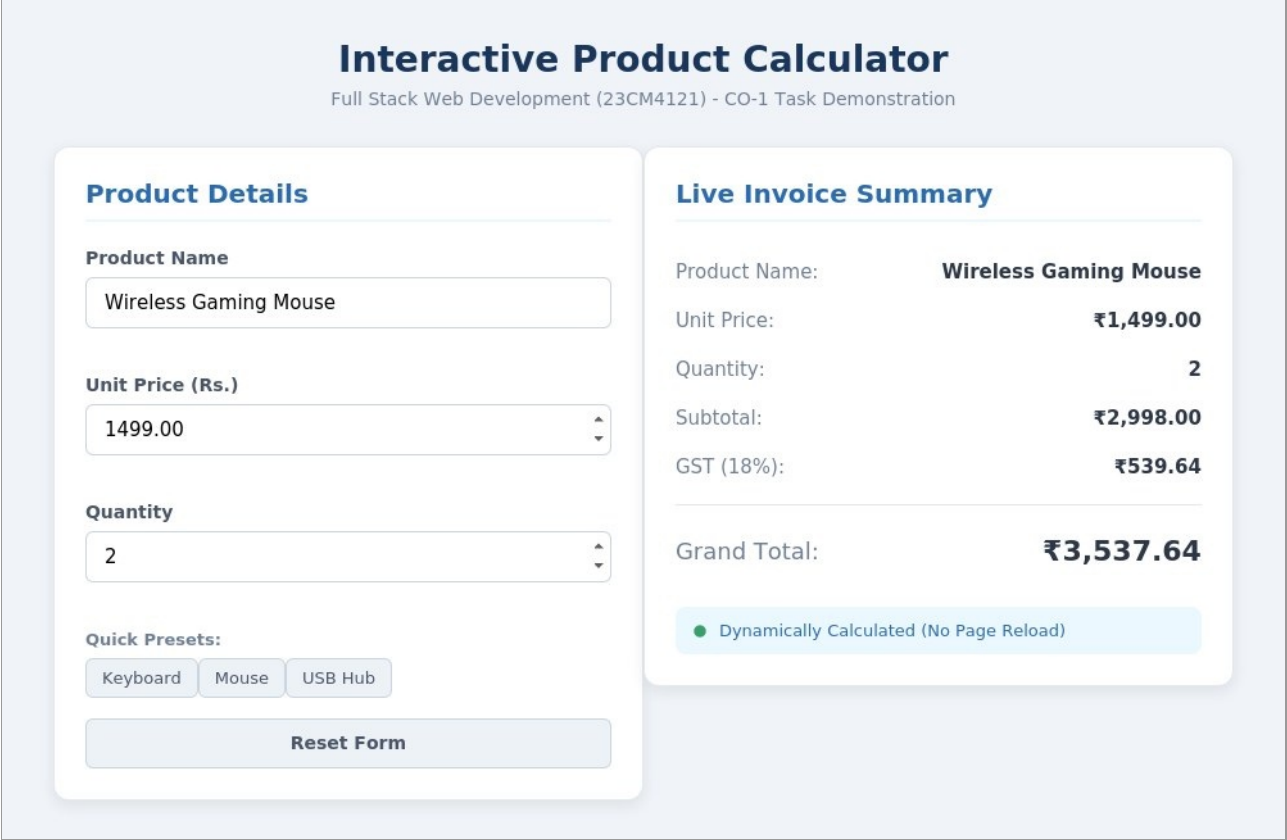


Figure 1: Full Webpage Output of Interactive Product Calculator

10. RESULT

Thus, the **Interactive Product Calculator** webpage was successfully developed, styled, and verified using HTML5, CSS3, and JavaScript. Event listeners (input and change) and dynamic DOM updates were implemented to calculate order totals in real time without refreshing the webpage.